Journeo - Bedienungsanleitung

Autoren: Tristan Häß, Robin Schiffmacher

Kurs: TINF23CS2

Veranstaltung: Advanced Software Engineering

Dozent: Philipp Fruck

Institution: DHBW Mannheim

Inhaltsverzeichnis

1 Journeo **1**
1.1 Überblick . . . 1
1.2 Architektur . . . 1
1.2.1 Komponenten . . . 1
1.2.2 Kommunikation und Datenfluss . . . 1
1.2.3 Deployment mit Docker Compose . . . 2
1.2.4 Architekturübersicht (vereinfacht) . . . 2

2 Guides **3**
2.1 User . . . 3
2.1.1 Voraussetzungen . . . 3
2.1.2 Nutzung . . . 3
2.1.3 Funktionen . . . 3
2.1.4 Troubleshooting . . . 4
2.2 Developer . . . 5
2.2.1 Voraussetzungen . . . 5
2.2.2 Installationen . . . 5
2.2.3 Entwicklung . . . 6
2.2.4 Tests . . . 10
2.2.5 Troubleshooting . . . 10
2.3 Admin . . . 11
2.3.1 Voraussetzungen . . . 11
2.3.2 Produktives Setup . . . 11
2.3.3 Daten . . . 11
2.3.4 Update . . . 12
2.3.5 Monitoring . . . 12
2.3.6 Troubleshooting . . . 12

1 Journeo

1.1 Überblick

Journeo ist eine webbasierte Anwendung, die es Nutzern ermöglicht, ihre Reisen zu planen und zu organisieren. Die Anwendung bietet Funktionen wie das Anlegen von Reisen, die Verwaltung von Reisedetails und die Planung. Die Zielgruppe von Journeo sind Privatpersonen und Reisebegeisterte, die ihre Reisen effizient und übersichtlich planen möchten. Die Anwendung ist so konzipiert, dass sie einfach zu bedienen ist und eine intuitive Benutzeroberfläche bietet, um die Verwaltung von Reisen zu erleichtern.

1.2 Architektur

Journeo ist als klassische 3-Schichten-Webanwendung umgesetzt und wird containerisiert betrieben. Die Anwendung besteht aus einem Frontend (Client/Präsentation), einem Backend (Logik/REST-API) und einer relationalen Datenbank (Persistenz).

1.2.1 Komponenten

- **Frontend (UI):** Statische Webanwendung (Vite/JavaScript), die dem Nutzer die Benutzeroberfläche bereitstellt. In Produktion wird das Frontend über einen Nginx-Webserver ausgeliefert.
- **Backend (API):** Python-Webservice (FastAPI), der die fachliche Logik kapselt und eine REST-API bereitstellt (u.a. für Reisen, Tage und Aktivitäten). Die API ist versioniert (`/api/v1/...`).
- **Datenbank:** PostgreSQL als persistente Datenhaltung. Das Backend greift über ein ORM (SQLAlchemy) auf die Daten zu. Das Schema wird in Modellen abgebildet.

1.2.2 Kommunikation und Datenfluss

Das Frontend kommuniziert ausschließlich per HTTP/JSON mit dem Backend. Der Zugriff auf die API erfolgt über einen automatisch generierten TypeScript-Client, der aus der OpenAPI-Spezifikation des Backends abgeleitet wird. Die Generierung erfolgt mithilfe des OpenAPI Generators. Dadurch stehen im Frontend typsichere und stets konsistente Schnittstellen für Endpunkte sowie Request- und Response-Modelle zur Verfügung. Änderungen an der API werden so frühzeitig im Build-Prozess sichtbar. Der generierte Code wird nicht manuell verändert, sondern bei Anpassungen der API erneut erzeugt. In der Container-Umgebung dient Nginx als Reverse Proxy. Aufrufe auf `/api` werden an das Backend (intern `backend:8000`) weitergeleitet, während alle übrigen Requests als statische

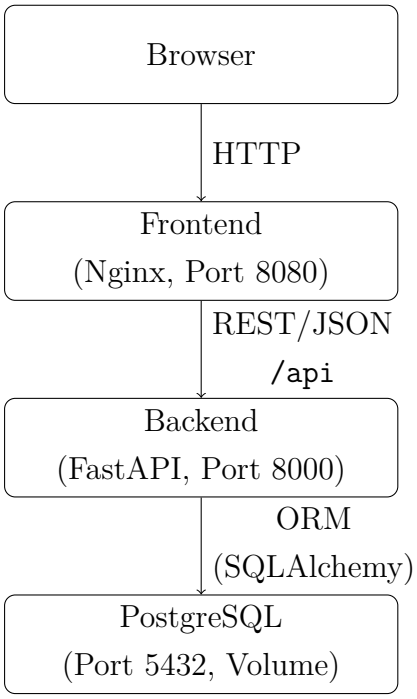
Frontend-Ressourcen bedient werden. Das Backend führt Validierungen durch, verarbeitet die Requests und persistiert Änderungen in der PostgreSQL-Datenbank.

1.2.3 Deployment mit Docker Compose

Für den Betrieb werden die Services über Docker Compose orchestriert:

- reproduzierbares Deployment über versionierte Images (Variable `VERSION`)
- persistente Datenhaltung über ein Docker-Volume (`se2data`)
- Healthchecks/Startreihenfolge: Backend startet zuverlässig, sobald die Datenbank erreichbar ist

1.2.4 Architekturübersicht (vereinfacht)



2 Guides

2.1 User

2.1.1 Voraussetzungen

Um die Anwendung zu nutzen, benötigt der Nutzer einen Webbrowser. Es sind keine speziellen Softwareinstallationen erforderlich, da die Anwendung vollständig webbasiert ist.

2.1.2 Nutzung

Im Browser muss die URL `http://IP-des-Servers:8080` aufgerufen werden, um die Anwendung zu erreichen. Im Anschluss hat der Nutzer Zugriff auf die Benutzeroberfläche und kann die angebotenen Funktionen nutzen.

2.1.3 Funktionen

Die Anwendung bietet folgende Möglichkeiten:

- Funktion 1: Dashboard mit Übersicht über die angelegten Reisen
- Funktion 2: Neue Reise anlegen
- Funktion 3: Reise bearbeiten und löschen
- Funktion 4: Detailansicht einer Reise

Funktion 1: Dashboard Das Dashboard zeigt eine Übersicht über alle angelegten Reisen. Es wird nicht differenziert, welcher Nutzer eine Reise angelegt hat und jeder kann alle Reisen einsehen. Falls noch keine Reisen angelegt wurden, wird eine leere Ansicht mit einem Hinweistext angezeigt. Sobald Reisen angelegt wurden, werden diese mit den nötigsten Informationen untereinander dargestellt. Von hier aus hat der Nutzer die Möglichkeit, entweder eine Reise zu bearbeiten, zu löschen oder die Detailansicht aufzurufen, um weitere Informationen zu erhalten.

Funktion 2: Neue Reise anlegen Über den Button `Reise hinzufügen` in der Topbar kann eine neue Reise angelegt werden. Es öffnet sich ein Formular, in dem die nötigen Informationen zur Reise eingegeben werden können (z.B. Name der Reise, Start- und Enddatum, Kosten und Beschreibung). Nach dem Speichern der Reise hat der Nutzer im Anschluss die Möglichkeit, bereits einzelne Tage für die Reise anzulegen, um die Reise weiter zu detaillieren. Innerhalb eines Tages können wiederum Aktivitäten angelegt werden, um den Tag weiter zu strukturieren und zu planen.

Funktion 3: Reise bearbeiten und löschen Über die Button-Symbole `Bearbeiten` und `Löschen` in der Übersicht kann die Reise bearbeitet oder gelöscht werden. Beim Bearbeiten können alle Informationen der Reise angepasst werden, beim Löschen wird die Reise mit allen zugehörigen Tagen und Aktivitäten unwiderruflich gelöscht.

Funktion 4: Detailansicht einer Reise Über das Button-Symbol `Ansicht` in der Übersicht kann die Detailansicht einer Reise aufgerufen werden. In der Detailansicht einer Reise werden alle Informationen der Reise sowie die angelegten Tage in einer Timeline dargestellt.

2.1.4 Troubleshooting

Falls die Anwendung nicht erreichbar ist, sollte zunächst geprüft werden, ob die URL korrekt eingegeben wurde und ob der Server, auf dem die Anwendung läuft, erreichbar ist. Sollte die Anwendung weiterhin nicht erreichbar sein, könnte es sein, dass der Server nicht ordnungsgemäß läuft oder dass es Netzwerkprobleme gibt. In diesem Fall sollte der Administrator kontaktiert werden, um das Problem zu beheben. Innerhalb der Anwendung kann der Nutzer zwar falsche oder unvollständige Daten eingeben, jedoch werden diese durch Validierungen abgefangen und es wird eine Fehlermeldung angezeigt, damit der Nutzer die Eingabe korrigieren kann.

2.2 Developer

2.2.1 Voraussetzungen

Um am Projekt mitzuarbeiten, braucht man eine übliche Development-Umgebung mit Git sowie Docker. Die Anwendung ist so konzipiert, dass eine lokale Entwicklung möglich ist, bei der alle Komponenten (Datenbank, Backend, Frontend) über Docker Compose gestartet werden können. Folgende Voraussetzungen sollten auf dem System installiert sein:

- Git
- Docker & Docker Compose
- Node.js & npm
- Python

Zu Beginn muss das Repository von GitHub geklont und in den Projektordner gewechselt werden:

1. `git clone https://github.com/tristan134/Software-Engineering-2.git`
2. `cd Software-Engineering-2`

2.2.2 Installationen

Für die Installationen ist es sinnvoll, ein dediziertes virtuelles Python-Environment zu nutzen, damit die Dependencies isoliert sind.

1. `pre-commit` ist verpflichtend und wird lokal installiert. Empfohlen ist die Installation über `pip`:

   - `pip install pre-commit`

   Danach sollte `pre-commit` im Repository initialisiert werden, damit die Hooks bei jedem Commit automatisch laufen:

   - `pre-commit install`

2. Neben den pre-commit checks ist auch `pytest` lokal notwendig. Dazu sollten in dem Python-Environment die Requirements aus dem Backend installiert werden:

   - `pip install -r backend/requirements.txt`

2.2.3 Entwicklung

Der Code liegt getrennt in `backend/` und `frontend/`. Um die Anwendung lokal zu starten existiert `docker-compose.local.yml`, das die Services direkt aus dem Quellcode baut. Für den produktiven Betrieb gibt es `docker-compose.yml`, welche die Images aus der Github-Container-Registry lädt und über die Variable `VERSION` pinnt.

Lokales Setup Am einfachsten ist es, lokal immer über Compose zu arbeiten. Dazu soll im Projektroot eine `.env`-Datei angelegt und die benötigten Variablen eingetragen werden (insbesondere `POSTGRES_VERSION`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST`, `POSTGRES_PORT`). Danach kann die Umgebung gebaut werden:

- `docker compose -f docker-compose.local.yml up --build`

Wenn alles läuft, ist das Backend unter `http://localhost:8000` erreichbar und das Frontend unter `http://localhost:8080`.

Frontend lokal entwickeln Für schnelleres Feedback kann das Frontend auch unabhängig vom Docker-Container als npm-Dev-Server gestartet werden. Dazu müssen zuerst die frontend-Dependencies installiert und danach der Dev-Server gestartet werden:

1. `cd frontend`
2. `npm install`
3. `npm run dev`

Codestruktur Der Quellcode ist strikt in `backend/` und `frontend/` getrennt. Beide Bereiche sind so aufgebaut, dass neue Features konsistent ergänzt werden können.

Das Backend (`backend/`, siehe Abbildung 1) ist als FastAPI-Anwendung strukturiert, wobei sich der Anwendungscode unter `backend/app/` befindet. Der Einstiegspunkt ist `app/app.py`, in dem die Anwendung initialisiert sowie Middleware und Router registriert werden. Die eigentliche API-Schicht liegt unter `app/api/`, wo insbesondere die Versionierung und die einzelnen Endpoints organisiert sind. Datenbankzugriff und Datenstrukturen sind in `app/db/` gebündelt. Hier befinden sich sowohl die Konfiguration und Session-Verwaltung als auch die SQLAlchemy-Modelle und die dazugehörigen Pydantic-Schemas. Tests werden getrennt vom Anwendungscode unter `backend/tests/` gehalten und mit Pytest umgesetzt. Bei Erweiterungen des Backends wird entlang dieser Struktur gearbeitet:

Neue Endpoints werden entweder als eigene Dateien unter `app/api/v1/endpoints/` angelegt oder in bestehende Ressourcen integriert und anschließend in der zentralen Router-Registrierung eingebunden. Für neue oder geänderte Payloads werden entsprechende Pydantic-Schemas definiert, sodass Validierung und OpenAPI-Dokumentation konsistent bleiben. Änderungen an der Persistenz erfolgen über die SQLAlchemy-Modelle und werden stets gemeinsam mit den zugehörigen Schemas umgesetzt. Für jede neue Funktionalität oder relevante Änderung wird ein Unit-Test unter `backend/tests/` ergänzt.

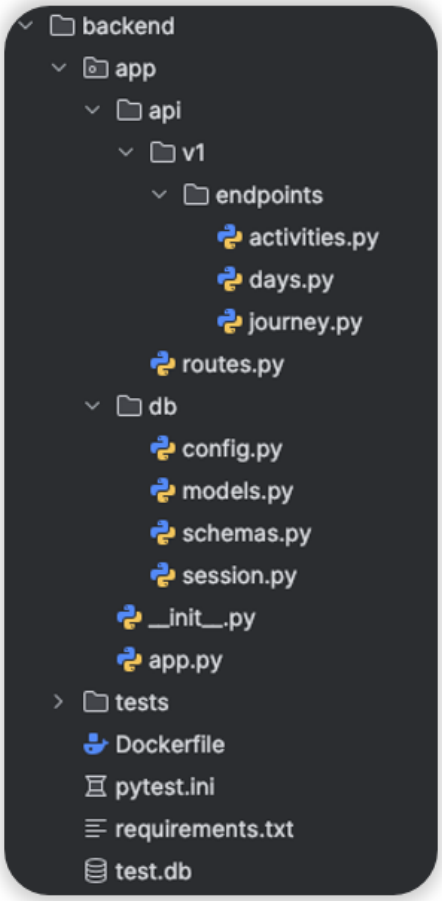


Abbildung 1: Codestruktur des backends

Das Frontend (`frontend/`, siehe Abbildung 2) ist als Vite-basiertes JavaScript-Projekt organisiert und liegt unter `frontend/src/`. Der Einstiegspunkt der Anwendung ist `src/main.js`. Die eigentliche Struktur folgt einem seitenbasierten Ansatz. Unter `src/pages/` befinden sich die einzelnen Views, die jeweils Rendering und zugehörige API-Aufrufe kapseln. Gemeinsame Funktionalität wie Layout oder Initialisierung ist in `src/pages/base.js` gebündelt, um Wiederverwendung zu fördern und Duplikation zu vermeiden.

Der Zugriff auf das Backend erfolgt über einen generierten API-Client unter `src/generated/`, der aus der OpenAPI-Spezifikation erzeugt wird und nicht manuell an-

gepasst wird. Änderungen an der API werden daher durch eine erneute Generierung dieses Clients in das Frontend übernommen.

Die Styles sind unter `src/css/` organisiert und nach Verantwortungsbereich getrennt. Globale Design-Definitionen befinden sich in `root.css`, wiederverwendbare Hilfsklassen in `utilities.css`, und seiten- bzw. feature-spezifische Styles in jeweils eigenen CSS-Dateien.
Bei Erweiterungen des Frontends werden neue Seiten unter `src/pages/` angelegt und in die bestehende Navigation integriert. Gemeinsame Logik wird dabei bewusst in `base.js` zentralisiert.
Änderungen an Backend-Endpunkten oder Datenmodellen erfordern eine erneute Generierung des OpenAPI-Clients, gefolgt von einer Anpassung der betroffenen Seiten an die aktualisierten Typen und Methoden. Anpassungen am Styling werden entsprechend ihrer Reichweite entweder global, als Utility oder seiten-spezifisch umgesetzt.

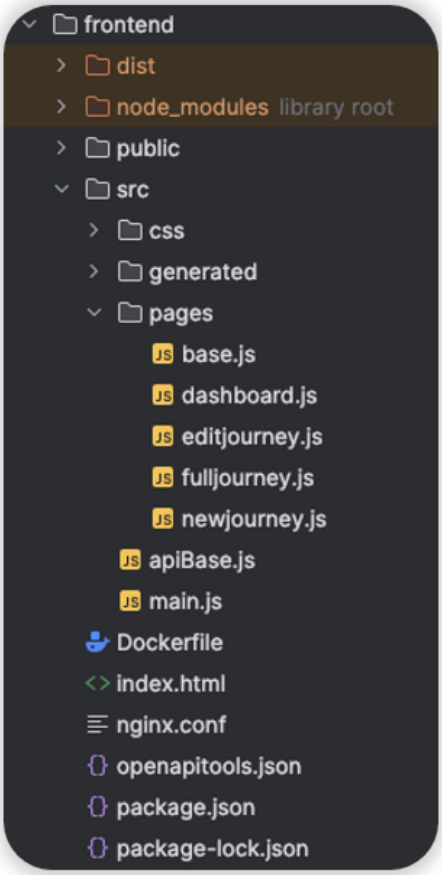


Abbildung 2: Codestruktur des frontends

Git-Workflows Änderungen passieren in Feature-Branches und werden dann über Pull Requests auf den `main`-Branch gemerged. Auf den `main` wird **nicht** force-gepusht. Zusätzlich gilt, dass Commits signiert werden sollen (Git commit signing via GPG oder SSH),

damit die Herkunft der Änderungen nachvollziehbar ist.
Bei jedem Pull Request laufen automatisch die GitHub Workflows und Tests durch (siehe Abbildung 3). Zudem muss jeder Pull Request von mindestens einem anderen Teammitglied reviewed und freigegeben werden. Erst wenn alle Checks erfolgreich sind, können die Änderungen über den Pull Request in den `main`-Branch gemerged werden.

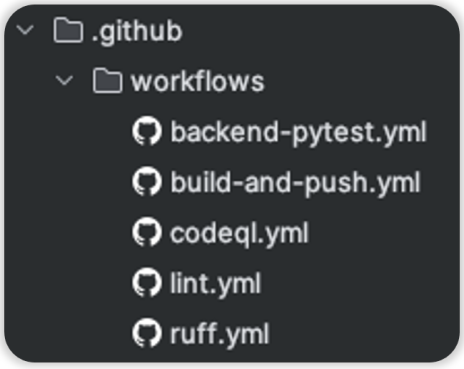


Abbildung 3: Github Workflows

Versionierung In `docker-compose.yml` werden produktive Images über `ghcr.io/...:${VERSION}` referenziert. Als Konvention entspricht `VERSION` einem Git-Tag (z.B. `v1.0.0`). Folgende Tag-Konventionen ist für die Versionierung vorgesehen:

- `vX.Y.Z`

Zu den Bedeutungen der einzelnen Stellen:

- `X`: Major-Version, z.B. bei Breaking Changes
- `Y`: Minor-Version, z.B. bei neuen Features ohne Breaking Changes
- `Z`: Patch-Version, z.B. bei Bugfixes

Sobald ein neues Feature fertig ist und der Pull Request durchgelaufen ist, soll ein neues Tag auf dem `main`-Branch erstellt und gepusht werden, damit die neue Version in Produktivsystemen deployt werden kann:

1. git checkout main
2. git pull
3. git tag vX.Y.Z
4. git push origin vX.Y.Z

Dazu läuft automatisch eine Action an, die Images aus dem Quellcode baut und in die Github-Container-Registry pusht.

2.2.4 Tests

Sobald an den Routen im Backend etwas verändert oder Neue hinzugefügt wurden, muss ein passender Pytest geschrieben werden. Die Tests sollen einheitlich in dem Ordner `backend/tests` liegen und im Namen die zu testende Endpoint Route enthalten. Spätestens vor einem Pull Request müssen die Tests erfolgreich lokal durchlaufen sein, damit Fehler frühst möglich zur Behebung erkannt werden können. Um die Tests lokal auszuführen, kann der folgende Befehl genutzt werden:

- `cd backend/tests && pytest`

2.2.5 Troubleshooting

Wenn die Datenbank nicht startet, sind es fast immer fehlende bzw. inkonsistente `POSTGRES_*`-Variablen oder ein kaputter Volume-Zustand. In letzter Konsequenz hilft `docker compose -f docker-compose.local.yml down -v` (Achtung: löscht Volumes und damit lokale Daten!). Wenn die benötigten Ports belegt sind, ist selbstständig zu prüfen, ob `8000` (Backend) oder `8080` (Frontend) bereits von anderen Diensten auf dem Computer verwendet werden.

2.3 Admin

Der produktive Betrieb ist containerbasiert und wird über `docker-compose.yml` beschrieben. Im Gegensatz zur lokalen Entwicklung wird in Produktion nicht aus dem Quellcode gebaut, sondern es werden versionierte Images aus der Github-Container-Registry geladen. Dadurch ist ein Deployment reproduzierbar. Dieselbe `VERSION` bedeutet dieselben Images.

2.3.1 Voraussetzungen

Benötigt wird ein Server bzw. eine VM mit *Docker* und *Docker Compose* sowie ausreichend Speicherplatz für die Daten. Ein minimaler Betrieb benötigt lediglich die Portfreigabe für den Port `8080` für das Frontend (UI). Der Backend-Port `8000` (API) muss nur dann zusätzlich nach außen geöffnet werden, wenn die API direkt von außen über Swagger UI erreichbar sein soll. Das Frontend ruft `/api` auf und der Nginx-Server vermittelt die Anfrage an das Backend über `backend:8000`.

2.3.2 Produktives Setup

Die `docker-compose.yml` wird auf den Server kopiert und eine `.env`-Datei daneben angelegt, in der die Konfiguration und Versionierung festlegt werden. Relevant sind insbesondere `VERSION` (Image-Tag), sowie die `POSTGRES_*`-Variablen. Besonders wichtig ist hier, dass sensible Informationen wie das Datenbankpasswort sicher gewählt werden sollen. Der Zugriff auf diesen Bereich sollte demnach auch streng kontrolliert werden. In einem Compose-Setup ist `POSTGRES_HOST` typischerweise der Service-Name `db` und `POSTGRES_PORT` ist `5432`.
Über `docker compose up -d` können die Container gestartet werden. Die Datenbank hat einen Healthcheck und das Backend startet erst zuverlässig, wenn die DB als healthy markiert ist. Im Anschluss ist die Anwendung über `http://IP-des-Servers:8080` erreichbar. Falls es nicht gewollt ist die IP-Adresse zum Aufrufen zu nutzen, kann auch ein DNS-Eintrag auf die IP-Adresse angelegt werden.

2.3.3 Daten

Die PostgreSQL-Daten liegen in dem Docker-Volume `se2data`. Dieses Volume ist der Grund, warum ein einfacher Redeploy ohne Datenverlust möglich ist. Wichtig: `docker compose down -v` löscht Volumes und damit die Daten, das sollte in Produktion nur in Ausnahmefällen und mit Backup passieren.

2.3.4 Update

Ein Update besteht im Wesentlichen daraus, die `VERSION` in der `.env`-Datei zu aktualisieren und anschließend die Services neu zu starten:

- `docker compose pull`
- `docker compose up -d`

Danach sollten die Logs kurz geprüft werden (z.B. `docker compose logs -f`). Falls eine Version fehlschlägt, kann mit dem vorherigen `VERSION`-Wert schnell zurückgerollt werden.

2.3.5 Monitoring

Ein minimalistisches Monitoring lässt sich über Docker-Bordmittel abdecken:

- `docker compose ps` für den Status
- `docker stats` für Ressourcen
- `docker compose logs` für Fehleranalyse.

2.3.6 Troubleshooting

Wenn das Backend die Datenbank nicht erreicht, kann es sein, dass der Service-Name in der docker-compose.yml nicht der gleiche ist wie die Variable `POSTGRES_HOST` in der `.env`-Datei. Wenn das Frontend nicht erreichbar ist, sind häufig Firewall/Portfreigaben die Ursache. Bei einem „broken“ Release ist ein Rollback durch Zurücksetzen von `VERSION` und erneutes `docker compose up -d` der schnellste Weg.